

Build Signals in Observability

Build signals connect deployments to production incidents.

Goal – Help engineers quickly answer: "What changed right before things broke?"

Most outages are caused by change

✗ WITHOUT BUILD SIGNALS

- Engineers manually compare logs
- Timelines become confusing
- Incident response slows down
- Root cause analysis unreliable

COMMON CAUSES

New deployment

Config update

Infra change

✓ WITH BUILD SIGNALS

- Deployments appear beside errors
- Engineers instantly see what changed
- Faster root cause investigation
- Better production visibility

Most incidents begin with a recent change.

Timeline

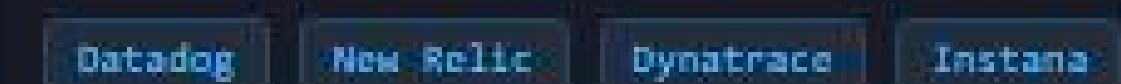


DURING INCIDENTS, ENGINEERS ASK:

- What changed?
- When did it change?
- Who deployed it?
- Did errors begin before or after deployment?

INDUSTRY TOOLS SOLVE THIS BY:

- Tracking deployments automatically
- Overlaying deploys onto metrics
- Correlating incidents with changes



FRAME 4 – WHAT BUILD SIGNALS ARE

Definition

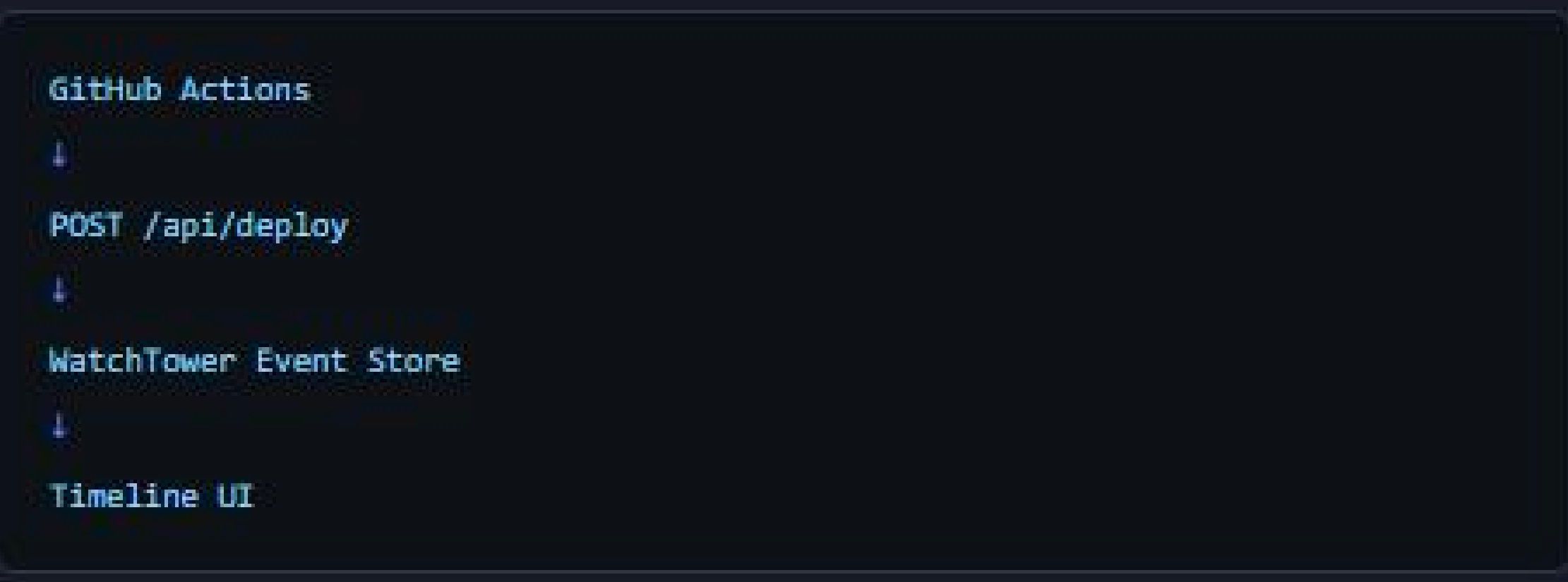
Build signals are structured deployment events emitted during CI/CD workflows.

MINIMUM USEFUL DEPLOYMENT DATA

timestamp Anchor for incident correlation	version / commit SHA Identifies exact deployed code	environment Distinguishes staging vs production
author Tracks who triggered deployment	status success / failure / in_progress	workflow_name Distinguishes deploy vs test workflows

These six fields answer: "Which deployment started the fire?"

Architecture



- No extra dependencies
- No webhook complexity
- Simple JSON payload
- Easy debugging

Deployment marker tooltip

Deploy **v1.3.2**
Author: **jsmith**
Workflow: **deploy-prod**
Status: **success**

"Did the issue begin before or after deployment?"
This is the core observability question.

4 core features

01 • DEPLOYMENT INGEST ENDPOINT

POST /api/deploy

Receives: version, timestamp, environment, author, workflow, status.
Source: Github Actions curl step.

02 • TIMELINE DEPLOYMENT MARKERS

Display deployments directly on error graphs and performance graphs.

03 • SHARED EVENT STORAGE

Store deploy events alongside errors, feedback, and performance samples.
Single event pipeline = simpler architecture.

04 • FAILED DEPLOY = INCIDENT SIGNAL

If **"status": "failure"**, surface immediately in the incident view.

Avoid in MVP

✗ Automatic correlation engines

Too complex for MVP scope

✗ Pipeline analytics dashboards

Not operationally necessary yet

✗ Github webhook infrastructure

Adds retry + verification complexity

✗ DORA metric computation

Store raw deployment data first

Build the observability foundation first.
Add intelligence later.

Suggested schema

```
{
  "type": "deploy", // keeps schema unified
  "timestamp": "2026-05-18T14:32:00Z",
  "version": "a3f9c21",
  "environment": "production",
  "author": "jsmith",
  "status": "success",
  "workflow": "deploy-production",
  "url": "https://github.com/org/repo/actions/runs/12345" // closes the loop
}
```

System safety rules

- Deploys must never fail because WatchTower is down – use

|| true
- UI rendering must remain fast – load deployment signals asynchronously
- Do not over-engineer scale – deployment volume is extremely low

Stability > Features

Observability Tool Research

Understanding the tools that help developers detect, diagnose, and resolve software issues.

Purpose – Enable developers to understand the internal state of their software to detect, diagnose, and resolve issues – improving reliability, minimizing downtime, and enhancing user experience.

Traditional Observability

User-Centric Observability

Popular tools overview

Traditional observability tools focus on server logs, metrics, and distributed tracing.

Sentry

ERROR MONITORING & PERFORMANCE TRACING

- Automatic error capture with stack traces
- Groups duplicate errors by fingerprint
- Links errors to specific releases and deployments
- Distributed tracing with spans across services
- Source maps for readable frontend stack traces

Datadog

INFRASTRUCTURE & FULL-STACK OBSERVABILITY

- Metrics, logs, and traces in one platform
- Host-level monitoring (CPU, memory, disk)
- APM with automatic service maps
- Log aggregation and search
- 500+ integrations

New Relic

APPLICATION PERFORMANCE MONITORING (APM)

- Agent-based instrumentation for app performance
- Transaction tracing and slow query detection
- Log management & infrastructure monitoring
- AI-powered anomaly detection

LogRocket

FRONTEND SESSION REPLAY & DEBUGGING

- Session replay tied to errors and network requests
- Console log and network request capture
- Redux/state inspection
- Error grouping with user context
- Funnel and conversion analysis

Honeycomb

HIGH-CARDINALITY EVENT ANALYSIS

- Slice and dice events by any attribute
- Locate unpredictable, hard-to-reproduce problems
- Infrastructure monitoring
- High-cardinality analysis

PostHog

PRODUCT ANALYTICS & USER BEHAVIOR

- Event tracking (clicks, page views, custom actions)
- Session replay and funnels
- Feature flags and A/B testing
- Heatmaps – self-hostable

FEATURE COMPARISON

Feature	Sentry	Datadog	New Relic	LogRocket	Honeycomb
Error Tracking & Crash Reporting	✓	✓	✓	✓	✓
APM	✓	✓	✓	✓	✓
Infrastructure Monitoring	✗	✓	✓	✗	✓
Log Management	✓	✓	✓	✓	✓
Session Replay	✓	✓	✗	✓	✗
Real User Monitoring (RUM)	✓	✓	✓	✓	✗
High-Cardinality Analysis	✗	✓	✓	✗	✓

Popular tools overview

User-centric tools focus on user experience and product behavior, bridging the gap between frontend behavior and backend performance.

PostHog

ALL-IN-ONE PRODUCT ANALYTICS

- 1,000,000 free events
- 5,000 free session recordings
- Feature flags and A/B testing
- Self-hostable

Amplitude

BEHAVIORAL ANALYSIS & AI ANALYTICS

- Complex product data analysis
- AI-powered analytics
- User journey visualization
- Feature flags

Mixpanel

EVENT-DRIVEN BEHAVIORAL ANALYTICS

- User retention and funnel tracking
- Cohort analysis
- Survey integrations (Survicate)
- User journey visualization

Heap

AUTOCAPTURE – TRACES EVERYTHING

- No manual event setup needed
- Retrospective analysis
- Historical data: 6 months (free) / 1 year (paid)
- Session replay and heatmaps as add-ons

Fullstory

DIGITAL EXPERIENCE & SESSION REPLAY

- Powerful session replays
- AI-driven user friction identification
- Heatmaps and surveys

Microsoft Clarity

FREE SESSION RECORDING & HEATMAPS

- Completely free
- Session recordings and heatmaps
- Event capture

FEATURE COMPARISON

Feature	PostHog	Amplitude	Mixpanel	Heap	Fullstory	MS Clarity
Event Capture	✓ 1M free events	✓	✓	✓ Auto – no setup 6mo history (free)	✓	✓ Free
Session Replay	✓ 5K free	✗	✗	✓ Add-on required	✓	✓ Free
Heat Maps	✓	✓	✗	✓ Add-on required	✓	✓ Free
Surveys	✓	✓	✓ 3rd party (Survicate)	✓ 3rd party (Chameleon)	✓	✗
Feature Flags	✓	✓	✗	✗	✗	✗
User Journey Visualization	✓	✓	✓	✓	✗	✗

What WatchTower should build

✓ REALISTIC TO INCLUDE

Error Tracking & Crash Reporting

Most important – understand where problems occur

Application Performance Monitoring

Latency, throughput, error rates – critical to UX

Log Management

Structured logs for dashboard insights

Real User Monitoring / Event Capture

Track browser events to pinpoint failing areas

Surveys

Collect user feedback to improve the software

✗ TOO COMPLEX FOR MVP

Infrastructure Monitoring

Requires server scripts injected into target software

Session Replay

Extra data collection + dashboard complexity

High-Cardinality Analysis

Requires preprocessing and aggregation pipelines

Heat Maps

Massive mouse event logging + display complexity

Feature Flags

Too much additional complexity

User Journey Visualization

Requires chaining multiple events across navigation

User Feedback Signals

The human side of build signals – marking when users reacted to what changed.

Core question – "Did this deploy upset people?" Build signals mark when software changed. User feedback signals mark when users reacted to that change.

Star Ratings

Feedback Forms

Bug Reports

CSAT

NPS

What are user feedback signals?

Measurable expressions of user sentiment collected directly from the product. Unlike infrastructure metrics (error rates, latency), these signals capture the **lived** experience of the user – confusion, frustration, delight, or abandonment.

MVP goal – Correlate these signals with build events to surface regressions in user experience automatically.

★ Star / Thumbs Rating Widget

INLINE MICRO-SURVEY · 1-5 STARS, THUMBS, OR EMOJI SCALE

WHAT IT CAPTURES

Immediate, low-friction sentiment tied to a specific moment or feature after a key user action

EXAMPLE TOOLS

- Hotjar
- Delighted
- Custom inline widget

ALERT CONDITION

Avg drops ≥ 1 point within 2h of build

Build v2.4.1 Rating avg drops: 4.2
+ 2.8 🚩 Signal: Possible UX regression

Each submission sent as an event with **timestamp**, **score**, and optional **page** or **feature** tag. WatchTower plots average rating over time with build markers overlaid.

📄 Feedback Forms (Open Text)

SHORT-FORM TEXT · "GIVE FEEDBACK" BUTTON, NPS FOLLOW-UP, EXIT INTENT

WHAT IT CAPTURES

Qualitative detail
numeric scores miss – users often name the exact broken thing

EXAMPLE TOOLS

- Typeform
- Intercom
- Native textarea widget

ALERT CONDITION

>10 negative keywords in 1h window post-build

Stored with **timestamp**, **text**, and optional **user_id** / **session_id**. WatchTower runs keyword/sentiment analysis – spike in "broken", "can't", "error", "slow" after a build triggers a flag.

🐛 Complaint / Bug Report Events

EXPLICIT HIGH-INTENT SIGNALS · IN-APP BUG REPORTS, SUPPORT TICKETS

WHAT IT CAPTURES

High-intent frustration. A user filing a bug is more upset than one clicking a low star.

EXAMPLE TOOLS

- Zendesk
- Linear
- GitHub Issues
- Custom report button

ALERT CONDITION

Volume 2× baseline within 4h of build. Severity weighting applied.

Each complaint is an event with **timestamp**, **category**, **severity**. A "data loss" complaint counts more than a "cosmetic glitch".

😊 CSAT – Customer Satisfaction Score

POST-INTERACTION SURVEY · "HOW SATISFIED WERE YOU?" SCORED 1-5 OR 1-10

WHAT IT CAPTURES

Feature-level satisfaction over time. More structured than open text, more granular than NPS.

EXAMPLE TOOLS

- Delighted
- SurveyMonkey
- Intercom CSAT

ALERT CONDITION

Feature CSAT drops $\geq 20\%$ post-deploy

Streamed as events with **timestamp**, **score**, **feature_context**. Drops in CSAT for a specific feature pinpoint *which part* of the release caused the regression.

📊 NPS – Net Promoter Score

"HOW LIKELY ARE YOU TO RECOMMEND US?" · SCORED 0-10 · SURVEYED MONTHLY/QUARTERLY

WHAT IT CAPTURES

Overall brand/product sentiment. Less real-time – better for release cycle retrospectives.

BUCKETS

- 0-6: Detractors
- 7-8: Passives
- 9-10: Promoters

ALERT CONDITION

Week-over-week drop ≥ 10 points after release

Score = % Promoters – % Detractors. Stored with **timestamp**. WatchTower shows NPS trend alongside a release history view – "NPS dropped 12 points in the week following the v2.4.0 launch".

Signal collection & alert conditions

Signal	Collection Method	WatchTower Hook	Alert Condition
Star Rating	Inline widget → event API	Rolling avg chart + build overlay	Avg drops ≥ 1 point within 2h of build
Feedback Form	Submit → event API	Keyword spike detector	>10 negative keywords in 1h window
Complaint / Bug	Ticket webhook	Volume spike chart	Volume 2× baseline within 4h of build
CSAT	Survey embed → event API	Per-feature score trend	Feature CSAT drops $\geq 20\%$ post-deploy
NPS	Survey tool webhook	Release retrospective view	Week-over-week drop ≥ 10 points

Proposed event shape

```
{
  "signal_type": "rating" | "feedback" | "complaint" | "csat" | "nps",
  "timestamp": "2025-06-01T14:32:00Z",
  "score": 2,
  "text": "The new checkout flow is broken on mobile",
  "feature_context": "checkout",
  "user_id": "u_abc123",
  "session_id": "sess_xyz",
  "build_id": "v2.4.1" // optional – can be inferred from timestamp
}
```

Key principles

- > **Lightweight** – Start with a single thumbs rating widget on 1-2 key screens.
first Timestamp + score alone is enough to prove the build-correlation concept.
- > **Anonymous is fine** – User ID is optional. Timestamp + score is sufficient for trend detection.
- > **Don't ask too often** – Triggered surveys (post-action) outperform timed popups. Rate-limit per user session.
- > **Backend requirement** – Ingestion endpoint that accepts signal events and stores them with millisecond timestamps for accurate build alignment.
- > **Build alignment logic** – WatchTower matches each signal event to the most recent build at or before its timestamp, powering the "signals since last deploy" view.

"Did this deploy upset people?"

Build signals + user feedback signals = the full picture.

Early-Stage Observability Needs & Strategy

Implementing observability is a delicate balancing act between speed, cost, and clarity.

Core insight – For small teams, observability must act as an enabler of velocity, not a tax on engineering time.

Bridging the gap between teams

In a startup, Product Managers and Support staff need visibility just as much as developers – observability isn't only for SREs.

Frontend ↔ Backend correlation

If a user clicks a button and nothing happens, backend logs might look healthy. True observability captures user friction at the edge (browser) and ties it to backend performance.

Contextual support

Support teams should move from asking "What browser are you using?" to simply looking up the user's session and seeing the exact error stack trace that occurred when they clicked submit.

High signal, low noise

Early-stage teams don't have the manpower to sift through terabytes of logs or monitor hundreds of granular dashboards.

✗ The "Dashboard Graveyard" Failure Mode

A common anti-pattern – building complex dashboards that track everything leads to nobody knowing what to look at during an incident. Focus only on actionable metrics.

✗ Alert Fatigue is an Existential Threat

For a team of 4-6 developers, constant pages for non-critical issues lead to burnout. If the team starts ignoring monitoring because it "cries wolf," the system has failed. Alerts must be reserved for situations requiring human intervention – user-facing 500 errors, broken checkout flows.

THE FOUR GOLDEN SIGNALS

If an anomaly doesn't impact one of these four pillars, it should not trigger a page.



Latency

How long requests take



Traffic

Volume of requests



Errors

Rate of failed requests



Saturation

How full the system is

What observability is hired to do

Observability tools aren't just "software monitors" – they complete specific developer jobs.

Job 1

"Alert me when there is a problem"

System Health / Metrics – real-time signal that something is wrong

Job 2

"Explain why the problem happened"

Context / Logs – the story behind the failure

Job 3

"Show me the blast radius"

Impact / User Feedback – how many users were affected and how

Job 4

"Tell me what changed"

Deployment Correlation – in a startup shipping multiple times a day, most incidents are caused by recent deployments. The tool must immediately answer: "Did the code we pushed 10 minutes ago cause this?"

Bridging the gap between teams

In a startup, Product Managers and Support staff need visibility just as much as developers – observability isn't only for SREs.

Frontend ↔ Backend correlation

If a user clicks a button and nothing happens, backend logs might look healthy. True observability captures user friction at the edge (browser) and ties it to backend performance.

Contextual support

Support teams should move from asking "What browser are you using?" to simply looking up the user's session and seeing the exact error stack trace that occurred when they clicked submit.

Avoiding the observability tax

✗ High-cardinality indexing at full scale

Storing 100% of network traces is financially unviable for early-stage startups. Rely on intelligent sampling and short data retention windows.

✗ High-maintenance tooling

If setting up and maintaining the observability stack takes more time than fixing actual bugs, the wrong tool was chosen. Setup must be as close to zero-config as possible.

If it takes more time to maintain than it saves – it's the wrong tool.

How WatchTower solves these exact startup needs

WatchTower's MVP is uniquely positioned by deliberately limiting its feature set to what early-stage teams actually need.

✓ Focus on "What Changed"

Directly correlating errors and user feedback to specific deployment hashes solves the most common startup question – "Did our release break this?"

✓ Unified Signals

Grouping Technical Errors, Performance Degradation, and User Feedback into a single pane of glass breaks down silos between Engineering, Support, and Product.

✓ Simplicity over Deep Tracing

Rather than complex distributed tracing, WatchTower provides immediate, high-level clarity needed to make a fast "Rollback vs. Fix" decision.

Speed · Cost · Clarity

The three forces every early-stage observability decision must balance.